JavaScript Numbers

A COMPLETE GUIDE

The Most Complete Full Stack Developer Roadmap

www.thedevspace.io

NUMBER CREATION

Decimal

```
1  123; // integer
2  -5; // negative number
3  3.14; // float
```

Simple base-10 numbers. JavaScript supports integers and floating point values.

Binary

Binary literals use `0b` prefix.

```
1  0b1010; // equals to decimal 10
```

Octal

```
1  0o755; // equals to decimal 493
```

Octal literals use `0o` prefix.

Hex

```
1  0xff; // equals to decimal 255
```

Hexadecimal literals use `0x` prefix.

Common for colors, bitmasks, and low-level values.

Swipe

Numeric Separator

Use underscore (`_`) to group digits for readability. Separators must appear between digits (not at ends or next to decimal point).

```
const oneM = 1_000_000;
const amount = 1_234_567.89;
```

BigInt

Large integers with arbitrary precision, created with an `n` suffix or `BigInt()`. Cannot mix BigInt with Number in arithmetic operation.

```
123n;
9007199254740993n; // beyond Number.MAX_SAFE_INTEGER

1n + 2; // TypeError: cannot mix BigInt and Number
```

Swipe

PARSING & CONVERTING

```
Number("12.3"); // 12.3
Number(true); // 1
Number(false); // 0
Number(null); // 0
Number(undefined); // NaN
Number(""); // 0
Number("   42  "); // 42
```

`Number(value)` converts to number. `null` converts to 0; `undefined` will be converted to `NaN`.

`parseInt(str, radix)` parses the string (`str`) to integer. Supports octal, hex, and binary numbers as well by specifying `radix`.

```
parseInt("08"); // decimal 8
parseInt("1010", 2); // binary 10
parseInt("755", 8); // octal 493
parseInt("ff", 16); // hex 255
```

```
parseFloat("3.14px"); // 3.14
```

`parseFloat(str)` parses string to floating number.

`num.toString(radix)` converts `num` to string. Supports octal, hex, and binary numbers as well by specifying `radix`.

```
(100).toString(); // decimal '100'
(10).toString(2); // binary '1010'
(493).toString(8); // octal '755'
(255).toString(16); // hex 'ff'
```

Swipe

IMPORTANT CONSTANTS

- `Number.MAX_VALUE` largest positive number.
- `Number.MIN_VALUE` smallest positive value.
- `Number.MAX_SAFE_INTEGER` largest integer safely represented.
- `Number.MIN_SAFE_INTEGER` smallest safe integer.
- `Number.EPSILON` difference between 1 and the next representable float, useful for tolerances.
- `NaN` Not-A-Number; result of invalid numeric operation and the only value not equal to itself.
- `Infinity` positive infinity.
- `-Infinity` negative infinity.

```
Number.MAX_VALUE;           // 1.7976931348623157e+308
Number.MIN_VALUE;           // 5e-324
Number.MAX_SAFE_INTEGER;    // 9007199254740991
Number.MIN_SAFE_INTEGER;    // -9007199254740991
Number.EPSILON;             // 2.220446049250313e-16

NaN;                        // NaN
Infinity;                   // Infinity
-Infinity;                  // -Infinity
```

Swipe

CHECKING VALUES

- `Number.isNaN(value)` reliable NaN check
- `Number.isFinite(value)` checks that value is a finite number
- `Number.isInteger(value)` returns true for integer numbers
- `Number.isSafeInteger(value)` checks if the number is within safe integer range

```
Number.isNaN(NaN); // true
Number.isNaN("NaN"); // false (no coercion)
Number.isFinite(1 / 0); // false
Number.isInteger(3.0); // true
Number.isSafeInteger(9007199254740992); // false
```

Swipe

ROUNDING & FORMATTING

`Math.round(x)` rounds x to the nearest integer; ties go toward +Infinity.

```
Math.round(1.5); // 2
Math.round(-1.5); // -1
```

```
Math.floor(1.9); // 1
Math.floor(-1.1); // -2
```

`Math.floor(x)` rounds down towards the largest integer less than or equal to x.

`Math.ceil(x)` rounds up towards the smallest integer greater than or equal to x.

```
Math.ceil(1.1); // 2
Math.ceil(-1.9); // -1
```

```
Math.trunc(1.9); // 1
Math.trunc(-1.9); // -1
```

`Math.trunc(x)` removes the fractional part.

`num.toFixed(n)` returns a string with exactly `n` digits after the decimal.

```
(1.2345).toFixed(2); // '1.23'
(1).toFixed(3); // '1.000'
```

Result is rounded and zero-padded.

Swipe

`num.toExponential(fractionDigits)` returns a string in scientific notation with the given digits after the decimal (argument optional).

```
(12345).toExponential(2); // '1.23e+4'
```

`num.toPrecision(precision)` returns a string with the specified number of significant digits; output may be fixed or exponential.

```
(1.2345).toPrecision(3); // '1.23'
(12345).toPrecision(3); // '1.23e+4'
```

`Intl.NumberFormat` locale-aware formatting for numbers, currencies, and percentages; configure with options.

```
new Intl.NumberFormat("en-US").format(12345.678); // '12,345.678'
new Intl.NumberFormat("de-DE", { style: "currency", currency: "EUR" }).format(
  1234.5
); // '1.234,50 €'
```

Swipe

MATH HELPERS

`Math.abs(x)` absolute value; returns non-negative magnitude.

```
Math.abs(-3); // 3
```

```
Math.max(1, 5, 3); // 5
```

`Math.max(...vals)` largest value among arguments; returns `-Infinity` if no args.

`Math.min(...vals)` smallest value among arguments; returns `Infinity` if no args.

```
Math.min(1, 5, 3); // 1
```

```
Math.pow(2, 10); // 1024
```

`Math.pow(x, y)` x raised to the power y (same as `x ** y`).

Swipe

`Math.sqrt(x)` square root; returns `NaN` for negative inputs.

```
Math.sqrt(9); // 3
```

```
Math.cbrt(-8); // -2
```

`Math.cbrt(x)` cube root (handles negatives).

`Math.random()` pseudo-random number in [0, 1); not cryptographically secure.

```
Math.random(); // 0.0 <= x < 1.0
```

```
Math.hypot(3, 4); // 5
```

`Math.hypot(...vals)` Euclidean norm (sqrt of sum of squares).

Useful for vector lengths and avoids overflow and underflow.

Swipe

`Math.sign(x)` sign of x: `1` (positive), `-1` (negative), `0` or `-0`, or `NaN` for non-numbers.

```
Math.sign(100); // 1
Math.sign(-100); // -1
Math.sign(-0); // -0
```

```
Math.imul(0xffffffff, 2); // -2
```

`Math.imul(a, b)` 32-bit integer multiplication with C-like wraparound; faster/accurate for 32-bit maths.

`Math.fround(x)` convert to 32-bit single-precision float (rounds to nearest float32).

```
Math.fround(1.337); // 1.3370000123977661
```

Swipe

QUICK REFERENCE TABLE

Category	**Methods / Notes**
Creation	123, 0b1010, 0o755, 0xFF, 1_000_000
Types	Number, BigInt (123n)
Convert	Number(x), +x, String(x), x.toString(radix)
Parse	parseInt(str, radix), parseFloat(str)
Check	Number.isNaN, Number.isFinite, Number.isInteger, Number.isSafeInteger
Rounding	Math.round, Math.floor, Math.ceil, Math.trunc, toFixed
Math	Math.max, Math.min, Math.pow, Math.sqrt, Math.hypot, Math.random
BigInt	123n, BigInt(str) (no mixing with Number)
Formatting	toFixed, toExponential, toPrecision, Intl.NumberFormat

Swipe